

ASSIGNMENT 11

Question 1: Implement isConnected() method in AdjListGraph/AdjacencyMatrix class (code written in class in last lecture).

The method should return true if all vertices in the graph are connected else return false.

I/P:-

```
adjMatGraph.addEdge(0, 1);  
adjMatGraph.addEdge(0, 2);  
adjMatGraph.addEdge(1, 2);  
adjMatGraph.addEdge(1, 3);  
adjMatGraph.addEdge(2, 3);
```

```
public boolean isConnected(int startVertex) {  
    if (vertexCount == 0) return true;  
    boolean[] isVisited = new boolean[vertexCount];  
    Queue<Integer> q = new LinkedList<>();  
    int visitedNodesCount = 0;  
    q.add(startVertex);  
    isVisited[startVertex] = true;  
    while(!q.isEmpty()) {  
        int vi = q.remove();  
        visitedNodesCount++;  
        for(int vj = 0; vj < vertexCount; vj++) {  
            if((adjMat[vi][vj] != 0) && (!isVisited[vj])) {  
                isVisited[vj] = true;  
                q.add(vj);  
            }  
        }  
    }  
    return visitedNodesCount == vertexCount;  
}
```

O/P:-

```
***** ADJACENCY MATRIX *****  
Is Connected:2  
Connected Count:false
```

Question 2: Implement `connectedComponentCount()` method in `AdjListGraph/AdjacencyMatrix` class (code written in class in last lecture). The method should return the number of connected components in a graph.

I/P:-

```
adjMatGraph.addEdge(0, 1);  
adjMatGraph.addEdge(0, 2);  
adjMatGraph.addEdge(1, 2);  
adjMatGraph.addEdge(1, 3);  
adjMatGraph.addEdge(2, 3);
```

@Override

```
public int connectedComponentCount() {  
    boolean[] isVisited = new boolean[vertexCount];  
    int count = 0;  
    for (int i = 0; i < vertexCount; i++) {  
        if (!isVisited[i]) {  
            count++;  
            dfsHelper(i, isVisited);  
        }  
    }  
    return count;  
}
```

O/P:-

```
***** ADJACENCY MATRIX *****  
Is Connected:2  
Connected Count:false
```

Question 3: Modify the Quicksort algorithm implementation to pick pivot number from a randomly generated position.

```
package in.cdac.Sort;
import java.util.Random;
import java.util.Arrays;
public class QuickSortRandom {
    public static void main(String[] args) {
        int[] data = {34, 7, 23, 32, 5, 62, 10, 15, 2};

        System.out.println("Original Array: " + Arrays.toString(data));
        quickSort(data, 0, data.length - 1);
        System.out.println("Sorted Array: " + Arrays.toString(data));
    }
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pIndex = partition(arr, low, high);

            quickSort(arr, low, pIndex - 1);
            quickSort(arr, pIndex + 1, high);
        }
    }
    private static int partition(int[] arr, int low, int high) {
        Random rand = new Random();
        int pivotIndex = rand.nextInt(high - low + 1) + low;

        swap(arr, pivotIndex, high);

        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr, i, j);
            }
        }

        swap(arr, i + 1, high);
        return i + 1;
    }
}
```

```
private static void swap(int[] arr, int i, int j) {  
    int temp = arr[i];  
    arr[i] = arr[j];  
    arr[j] = temp;  
}  
}
```

O/P:-

Original Array: [34, 7, 23, 32, 5, 62, 10, 15, 2]
Sorted Array: [2, 5, 7, 10, 15, 23, 32, 34, 62]